In [1]:

```
# ================================================================
# 1. IMPORT LIBRARIES
# ================================================================

import pandas as pd
import numpy as np

# Train and test split
from sklearn.model_selection import train_test_split

# Feature scaling
from sklearn.preprocessing import StandardScaler

# Recursive Feature Elimination
from sklearn.feature_selection import RFE

# Regression algorithms
from sklearn.linear_model import LinearRegression
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

# Evaluation metric
from sklearn.metrics import r2_score
```

In [2]:

```
# ================================================================
# 2. RECURSIVE FEATURE ELIMINATION
# ================================================================

def recursive_feature_elimination(indep_X, dep_Y, n):

    # Create Linear Regression estimator
    estimator = LinearRegression()

    # Create RFE object
    selector = RFE(
        estimator=estimator,
        n_features_to_select=n
    )

    # Fit RFE on the data
    fit1 = selector.fit(indep_X, dep_Y)

    # Select the required features
    selectk_features = fit1.transform(indep_X)

    # Return selected features
    return selectk_features
```

In [3]:

```
# ================================================================
# 3. TRAIN TEST SPLIT + STANDARD SCALING
# ================================================================

def split_scalar(indep_x, dep_y):
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    # Split the data into training and testing data
    x_train, x_test, y_train, y_test = train_test_split(
        indep_x,
        dep_y,
        test_size=0.25,
        random_state=0
    )

    # Create StandardScaler object
    sc = StandardScaler()

    # Fit scaler only on training data
    x_train = sc.fit_transform(x_train)

    # Apply the same scaling to test data
    x_test = sc.transform(x_test)

    return x_train, x_test, y_train, y_test
```

In [4]:

```
# ============================================================
# 4. R2 SCORE PREDICTION
# ============================================================

def r2_prediction(regressor, X_test, y_test):

    # Predict test data
    y_pred = regressor.predict(X_test)

    # Calculate R2 score
    r2 = r2_score(y_test, y_pred)

    return r2
```

In [5]:

```
# ============================================================
# 5. LINEAR REGRESSION
# ============================================================

def Linear(X_train, y_train, X_test, y_test):

    regressor = LinearRegression()

    regressor.fit(X_train, y_train)

    r2 = r2_prediction(
        regressor,
        X_test,
        y_test
    )

    return r2
```

In [6]:

```
# ============================================================
# 6. SVM REGRESSION
# ============================================================
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
def svm_regression(X_train, y_train, X_test, y_test):

    regressor = SVR(kernel='rbf')

    regressor.fit(X_train, y_train)

    r2 = r2_prediction(
        regressor,
        X_test,
        y_test
    )

    return r2
```

In [7]:

```
# ================================================================
# 7. DECISION TREE REGRESSION
# ================================================================

def Decision(X_train, y_train, X_test, y_test):

    regressor = DecisionTreeRegressor(
        random_state=0
    )

    regressor.fit(X_train, y_train)

    r2 = r2_prediction(
        regressor,
        X_test,
        y_test
    )

    return r2
```

In [8]:

```
# ================================================================
# 8. RANDOM FOREST REGRESSION
# ================================================================

def random(X_train, y_train, X_test, y_test):

    regressor = RandomForestRegressor(
        n_estimators=10,
        random_state=0
    )

    regressor.fit(X_train, y_train)

    r2 = r2_prediction(
        regressor,
        X_test,
        y_test
    )

    return r2
```

In [9]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
# ============================================================
# 9. CREATE RESULT DATAFRAME
# ============================================================

def rfe_regression(
    acclinear,
    accsvr,
    accdes,
    accrf
):

    dataframe = pd.DataFrame(
        index=['RFE'],
        columns=[
            'Linear',
            'SVR',
            'Decision',
            'Random'
        ]
    )

    for number, index in enumerate(dataframe.index):

        dataframe.loc[index, 'Linear'] = acclinear[number]
        dataframe.loc[index, 'SVR'] = accsvr[number]
        dataframe.loc[index, 'Decision'] = accdes[number]
        dataframe.loc[index, 'Random'] = accrf[number]

    return dataframe
```

In [10]:

```
# ============================================================
# 10. LOAD DATASET
# ============================================================

dataset1 = pd.read_csv(
    "50_startups_dataset.csv",
    index_col=None
)

dataset1.head()
```

Out[10]:

	Unnamed: 0	R&D Spend	Administration	Marketing Spend	State	Profit
0	0	165349.30	136897.90	471784.20	New York	192261.93
1	1	162597.80	151377.69	443898.63	California	191792.16
2	2	153441.61	101145.65	407934.64	Florida	191050.49
3	3	144372.51	118671.95	383199.72	New York	182902.09
4	4	142107.44	91391.87	366168.52	Florida	166188.04

In [11]:

```
# ============================================================
# 11. CREATE DUMMY VARIABLES
# ============================================================
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
df2 = dataset1

df2 = pd.get_dummies(
    df2,
    drop_first=True
)
```

In [13]:

```
# ================================================================
# 12. SEPARATE X AND Y
# ================================================================

indep_X = df2.drop(
    'Profit',
    axis=1
)

dep_Y = df2['Profit']
```

In [14]:

```
# ================================================================
# 13. SELECT TOP 5 FEATURES USING RFE
# ================================================================

rfe = recursive_feature_elimination(
    indep_X,
    dep_Y,
    5
)
```

In [15]:

```
# ================================================================
# 14. CREATE EMPTY LISTS
# ================================================================

acclinear = []
accsvr = []
accdes = []
accrf = []
```

In [16]:

```
# ================================================================
# 15. TRAIN TEST SPLIT
# ================================================================

X_train, X_test, y_train, y_test = split_scalar(
    rfe,
    dep_Y
)
```

In [17]:

```
# ================================================================
# 16. RUN MACHINE LEARNING MODELS
# ================================================================

# Linear Regression
acc_linear = Linear(
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
    X_train,
    y_train,
    X_test,
    y_test
)

acclinear.append(acc_linear)


# SVR
acc_s = svm_regression(
    X_train,
    y_train,
    X_test,
    y_test
)

accsvr.append(acc_s)
# Decision Tree
acc_d = Decision(
    X_train,
    y_train,
    X_test,
    y_test
)

accdes.append(acc_d)


# Random Forest
acc_r = random(
    X_train,
    y_train,
    X_test,
    y_test
)

accrf.append(acc_r)
```

In [18]:

```
# ============================================================
# 17. CREATE FINAL RESULT
# ============================================================

result = rfe_regression(
    acclinear,
    accsvr,
    accdes,
    accrf
)
```

In [19]:

```
# ============================================================
# 18. DISPLAY RESULT
# ============================================================

result
```

Out[19]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

	Linear	SVR	Decision	Random
RFE	0.947313	-0.064129	0.97731	0.991572

In [ ]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF